ING. CARLOS JOSUE

Ingeniero Full-Stack | Especialista en Java y React, Microservicios, AWS y Arquitecturas Reactivas. Experto en Clean Code y optimización de costos operativos.

CONTACTO

+52 963 158 5303 | carlosjosuelopezsolano98@gmail.com | https://carlosportfolio-five.vercel.app

EDUCACION

Ingeniero en Sistemas Cumputacionales | Universidad: Instituto Tecnologico de Comitan

IDIOMAS

English: B2 Español:Nativo

SKILLS

Backend: AWS (CDK, Terraform), Java,Spring Boot ,Docker,Kubernetes,Kafka,Spring web flux,Prometheus, Grafana,JUnit,Github,ArgoCd,Backstage
Patrones de diseño,Swagger
Frontend:React js,Next js,Tailwind,TypeScript,ReactQuery,Redux,Axios,JavaScript
Datos:Redis,Elastic search, PostgreSQL, MySQL, MongoDB

PROJECTOS Y EXPERIENCIA

BrainTrust: Intelligent LMS with Academic Integrity

Tecnologias: Java 25, Spring Boot 3.6, Next.js 15, React 19, Gemini AI Agents, AWS, PostgreSQL, Docker, TypeScript.

- Desarrollé un LMS (Learning Management System) completo, similar a Moodle, para la gestión integral de estudiantes, asignaturas, entregas y calificaciones en un entorno universitario.
- Potencié la plataforma con Agentes de IA (Google Gemini), creando un sistema de auditoría automática que detecta si las tareas fueron generadas por IA o si existe plagio entre estudiantes mediante análisis semántico.
- Implementé el núcleo del sistema bajo Arquitectura Hexagonal y DDD, permitiendo que la lógica educativa sea totalmente independiente de los servicios de IA y de la base de datos.
- Optimicé la concurrencia utilizando Virtual Threads (Java 25), permitiendo procesar ~600 solicitudes diarias y llamadas asíncronas a la IA sin bloquear recursos, manteniendo tiempos de respuesta mínimos.
- Construí una interfaz de usuario moderna con Next.js 15 y React 19, utilizando Server Components para una navegación instantánea entre cursos y paneles de administración de estudiantes.
- Logré una eficiencia de costos extrema en AWS, operando la plataforma con un presupuesto de ~$250 MXN mensuales, gracias al tuning de la JVM y una arquitectura orientada a la eficiencia de recursos.
- Diseñé el esquema de base de datos en PostgreSQL para manejar relaciones complejas de inscripciones y kárdex académico, asegurando la integridad de los datos bajo estándares profesionales.
- Aseguré la calidad del software con Clean Code y Gitflow, documentando cada endpoint con Swagger/OpenAPI para una integración fluida entre el frontend y los microservicios.

NeoBank — Ecosistema Bancario Nativo de la Nube

Tecnologías: Java 25 (virtual thread), Spring Boot 3, AWS (EC2, Lambda, RDS, DynamoDB), Python, Cognito, SQS, SNS, Rekognition

- Arquitectura Híbrida en la Nube: Diseñé un backend de alta disponibilidad que combina un monolito Spring Boot (Auth/KYC) en EC2 con un conjunto de microservicios serverless (8 Lambdas) para el procesamiento de transacciones.
- Motor FinTech Orientado a Eventos:Desarrollé un pipeline de transacciones desacoplado usando SQS y DynamoDB, logrando el procesamiento paralelo de detección de fraude, contabilización y notificaciones en <400ms.
- Cumplimiento Impulsado por IA: Integré AWS Rekognition para la verificación automatizada de identidad KYC, reduciendo la carga manual mediante la validación biométrica multipunto.
- Seguridad Crítica para la Misión: Implementé transacciones ACID con bloqueo a nivel de fila de PostgreSQL para prevenir el doble gasto, junto con JWT basado en Cognito y gestión de sesiones mediante cookies HttpOnly.
- Fraude y Analítica en Tiempo Real:Construí un sistema automatizado de puntuación de riesgo en Python y un panel de métricas personalizado de CloudWatch para monitorear el estado de las transacciones y los KPIs del negocio en tiempo real.

TradeFlow — Mercado de Microservicios Orientado a Eventos

Tecnologías: Java 21, Spring Boot 3.2, Quarkus (GraalVM Native), Kafka, Kubernetes, PostgreSQL, MongoDB, Redis, Elasticsearch, Dagger, ArgoCD,Backstage

- Orquestación SAGA Distribuida: Diseñé una plataforma de comercio de alta escala con más de 10 microservicios, coordinando ciclos de vida de pedidos complejos mediante un orquestador SAGA central y un bus de comandos Kafka.
- Streaming de Eventos Confiable: Implementé el Patrón de Buzón de Salida Transaccional (Transactional Outbox) y Event Sourcing para asegurar cero pérdida de datos y auditabilidad completa en los almacenes PostgreSQL y MongoDB distribuidos.
- Prevención de Fraude Impulsada por IA:Desarrollé un motor de puntuación de riesgo en tiempo real usando Spring AI con embeddings vectoriales y similitud coseno, compilado a GraalVM Native para una latencia inferior a 10ms.
- Pagos de Alto Rendimiento:Construí una pasarela de pago reactiva y no bloqueante usando Quarkus Mutiny y Stripe, aplicando Claves de Idempotencia estrictas para garantizar la corrección financiera.
- Infraestructura Resiliente y GitOps:Automatizé un ecosistema sin tiempo de inactividad en Kubernetes con ArgoCD para GitOps, Resilience4j (Circuit Breakers/Bulkheads) y Dagger para pipelines de CI/CD portables.

Sistema de Microservicios: Food Ordering System (Alta Disponibilidad)

Tecnologías: Java 17, Spring Boot, Kafka, PostgreSQL, Redis, Docker, Kubernetes, Prometheus, Grafana.

- Diseñé un sistema distribuido tolerante a fallos capaz de procesar más de 10,000 pedidos concurrentes, implementando el patrón SAGA para gestionar transacciones distribuidas complejas.
- Arquitecté la solución bajo principios de Hexagonal Architecture, DDD y CQRS, utilizando Event Sourcing para asegurar la trazabilidad absoluta y consistencia de los datos.
- Orquesté la comunicación asíncrona entre 4 microservicios mediante Apache Kafka, logrando una disponibilidad del 99.9% y tiempos de respuesta inferiores a 200ms.
- Desarrollé mecanismos de compensación automática dentro del flujo de pedidos, reduciendo el tiempo de recuperación ante errores en un 85%.
- Implementé CDC (Change Data Capture) con Debezium y PostgreSQL, garantizando la sincronización de datos en tiempo real entre los diferentes servicios del ecosistema.
- Fortalecí la resiliencia del sistema de pagos aplicando patrones de Circuit Breaker y Exponential Backoff, evitando fallos en cascada durante picos de tráfico.
- Optimicé el throughput a 5,000+ pedidos por minuto mediante estrategias de Auto-scaling y balanceo de carga en un clúster de Kubernetes.
- Establecí un stack de observabilidad integral con Prometheus y Grafana, diseñando dashboards personalizados para el monitoreo proactivo de métricas de negocio y salud del sistema.
- Automaticé el ciclo de despliegue con Docker y CI/CD, asegurando lanzamientos con zero-downtime en entornos de producción.

Soccer Tournament Management System: Plataforma Integral de Gestión Deportiva

Tecnologías: React 19, Next.js 15, Spring Boot 3.x, TypeScript, Redux, React Query, Tailwind CSS, GSAP, Supabase, PostgreSQL, Docker.

- Desarrollé una plataforma Full-Stack de alto rendimiento para la administración de ligas de fútbol, permitiendo la gestión automatizada de torneos, equipos y estadísticas en tiempo real.
- Implementé un sistema de autenticación robusto utilizando JWT con Refresh Tokens y HttpOnly Cookies, garantizando una sesión segura y protegida contra ataques XSS/CSRF.
- Diseñé una arquitectura de seguridad basada en roles (RBAC), diferenciando dinámicamente las capacidades de administración, árbitros y equipos tanto en el Backend como en una UI adaptativa.
- Automaticé la lógica de programación de encuentros (Match Scheduling) y el seguimiento financiero, incluyendo el rastreo de deudas de jugadores y la gestión de pagos a árbitros.
- Optimicé la experiencia de usuario (UX) integrando animaciones 3D con GSAP y soporte completo para Dark/Light mode, utilizando las últimas funcionalidades de React 19 y Next.js 15.
- Gestión de estado avanzada y eficiencia de datos: Utilicé Redux para el estado global y React Query para la sincronización asíncrona con la API, reduciendo la latencia de carga.
- Procesamiento de eventos en tiempo real: Implementé el registro de goles, tarjetas y sustituciones con actualización inmediata de tablas de posiciones y estadísticas de jugadores.
- Infraestructura escalable con Docker y Supabase: Configuré el despliegue mediante contenedores y utilicé PostgreSQL para garantizar la integridad referencial de los datos deportivos complejos.

CloudMart — Plataforma de Comercio Electrónico Serverless en AWS

Tecnologías:Java 21, Spring Boot 3.3, AWS (Lambda, SQS, SNS, RDS, DynamoDB, Cognito, S3), Python, Terraform

- Núcleo Serverless Orientado a Eventos:Diseñé un mercado altamente escalable utilizando una arquitectura desacoplada donde Spring Boot orquesta la lógica de negocio y AWS Lambda maneja tareas especializadas como cálculos de envío.
- Pipeline de Mensajes Asíncronos:Integré SQS y SNS para gestionar los ciclos de vida de los pedidos y las notificaciones, asegurando la resiliencia del sistema y la escalabilidad horizontal durante ráfagas de alto tráfico.
- Infraestructura como Código (IaC):Arquitecté y desplegué todo el ecosistema en la nube (VPC, RDS, API Gateway) usando Terraform, permitiendo entornos reproducibles y aprovisionamiento automatizado de recursos.
- Orquestación de Identidad y Seguridad:Implementé autenticación de nivel empresarial a través de AWS Cognito con Lambda Authorizers, asegurando las APIs REST con tokens JWT y cookies HttpOnly.
- Estrategia de Datos Políglota:Optimicé la persistencia combinando MySQL RDS para transacciones compatibles con ACID con DynamoDB para búsquedas de alta velocidad de sesiones/carritos y S3 para la entrega global de activos.